

学号：202300130183	姓名：宋浩宇
实验名称：Transformer 基础实验——自注意力机制与编码结构理解	
1. 实验目的	
1. 理解 Transformer 模型的基本结构。 2. 掌握自注意力机制（Self-Attention）的核心思想。 3. 理解 Query、Key、Value 的作用。 4. 学习简单实现自注意力计算过程。 5. 分析 Transformer 相比传统模型（RNN）的优势。	
2. 实验原理	
Transformer 是现代自然语言处理的重要基础模型，其核心是自注意力机制（Self-Attention）。对于一个句子： 我 爱 自然语言处理 每个词在计算表示时，不仅依赖自身，还会“关注”其他词。 自注意力的基本计算流程为： $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) V$ 其中： Q（Query）：查询向量 K（Key）：键向量 V（Value）：值向量 通过这种机制，模型可以捕捉长距离依赖关系。	
3. 实验步骤与代码	
实验步骤：	
1. 对句子进行分词并构造词向量 2. 初始化 Q、K、V 矩阵 3. 计算 QK^T 4. 进行缩放与 softmax 5. 得到注意力权重 6. 与 V 相乘得到输出 7. 输出注意力矩阵并分析	
代码：	
<pre># -*- coding: utf-8 -*- import math import numpy as np DATA_PATH = r"./exp-6.1-data.txt" RNG_SEED = 42 D_MODEL = 16 def softmax(x, axis=-1): x = x - np.max(x, axis=axis, keepdims=True) e = np.exp(x) return e / np.sum(e, axis=axis, keepdims=True) def read_sentences(path):</pre>	

```

with open(path, "r", encoding="utf-8") as f:
    return [ln.strip() for ln in f if ln.strip()]
def tokenize(sentence):
    s = sentence.strip()
    if not s:
        return []
    if " " in s:
        return [t for t in s.split() if t]
    return list(s)
def stable_token_embeddings(tokens, d_model, seed):
    rows = []
    for t in tokens:
        sub = int(seed) + sum((k + 1) * ord(c) for k, c in
enumerate(t))
        rng = np.random.default_rng(sub % (2**32))
        rows.append(rng.normal(size=(d_model,)).astype(np.float64
))
    return np.stack(rows, axis=0) * 0.15
def self_attention(tokens, d_model=D_MODEL, seed=RNG_SEED):
    rng = np.random.default_rng(seed)
    n = len(tokens)
    if n == 0:
        raise ValueError("空词序列")
    x = stable_token_embeddings(tokens, d_model, seed)
    wq = rng.normal(size=(d_model, d_model)).astype(np.float64) *
0.1
    wk = rng.normal(size=(d_model, d_model)).astype(np.float64) *
0.1
    wv = rng.normal(size=(d_model, d_model)).astype(np.float64) *
0.1
    q = x @ wq
    k = x @ wk
    v = x @ wv
    scale = math.sqrt(d_model)
    scores = (q @ k.T) / scale
    attn = softmax(scores, axis=-1)
    out = attn @ v
    return attn, out, x
def print_matrix(title, mat, row_labels, col_labels):
    print(f"\n--- {title} ---")
    print("列(作为 key 位置):", " ".join(col_labels))
    for i, row_label in enumerate(row_labels):
        row = " ".join(f"{mat[i, j]:.6f}" for j in
range(mat.shape[1]))

```

```

        print(f" {row_label}: {row}")
def top_pairs(attn, tokens, top_k=5):
    n = len(tokens)
    pairs = []
    for i in range(n):
        for j in range(n):
            pairs.append((attn[i, j], i, j))
    pairs.sort(reverse=True, key=lambda t: t[0])
    out = []
    seen = set()
    for w, i, j in pairs:
        key = (i, j)
        if key in seen:
            continue
        seen.add(key)
        out.append((w, tokens[i], tokens[j], i, j))
        if len(out) >= top_k:
            break
    return out
def main():
    np.random.seed(RNG_SEED)
    print(f"数据文件: {DATA_PATH}  d_model={D_MODEL}  随机种子={RNG_SEED}")
    sentences = read_sentences(DATA_PATH)
    for si, sent in enumerate(sentences, 1):
        toks = tokenize(sent)
        print(f"\n{'=' * 12} 句子 {si} {'=' * 12}")
        print(f"原文: {sent}")
        print(f"分词: {' / '.join(toks)}")
        attn, out, emb = self_attention(toks, d_model=D_MODEL,
seed=RNG_SEED + si)
        print_matrix("注意力权重矩阵(行: query 词, 列: key 词)",
attn, toks, toks)
        print("\n--- 每个 query 词对应的输出向量范数 ---")
        for t, row in zip(toks, out):
            print(f" {t}: L2={np.linalg.norm(row):.6f}")
        print("\n--- 按权重从高到低的前若干词对(query 行对 key 列) ---")
        for rank, (w, q_tok, k_tok, i, j) in enumerate(
            top_pairs(attn, toks, top_k=min(8, len(toks) *
len(toks))), 1
        ):
            print(f" {rank}. query={q_tok} 对 key={k_tok} 权重={w:.6f} (i={i}, j={j})")

```

```
if __name__ == "__main__":  
    main()
```

4. 实验结果（图表与输出）

输出结果为：

===== 句子 1 =====

原文: 我 爱 自然语言处理

分词: 我 / 爱 / 自然语言处理

--- 注意力权重矩阵（行: query 词, 列: key 词） ---

列(作为 key 位置): 我 爱 自然语言处理

我: 0.332701 0.333283 0.334016

爱: 0.334920 0.332424 0.332656

自然语言处理: 0.332053 0.333425 0.334523

--- 每个 query 词对应的输出向量范数 ---

我: L2=0.091177

爱: L2=0.090762

自然语言处理: L2=0.091309

--- 按权重从高到低的前若干词对（query 行对 key 列） ---

1. query=爱 对 key=我 权重=0.334920 (i=1, j=0)

2. query=自然语言处理 对 key=自然语言处理 权重=0.334523 (i=2, j=2)

3. query=我 对 key=自然语言处理 权重=0.334016 (i=0, j=2)

4. query=自然语言处理 对 key=爱 权重=0.333425 (i=2, j=1)

5. query=我 对 key=爱 权重=0.333283 (i=0, j=1)

6. query=我 对 key=我 权重=0.332701 (i=0, j=0)

7. query=爱 对 key=自然语言处理 权重=0.332656 (i=1, j=2)

8. query=爱 对 key=爱 权重=0.332424 (i=1, j=1)

===== 句子 2 =====

原文: 机器学习 很 有趣

分词: 机器学习 / 很 / 有趣

--- 注意力权重矩阵（行: query 词, 列: key 词） ---

列(作为 key 位置): 机器学习 很 有趣

机器学习: 0.333991 0.333475 0.332534

很: 0.335423 0.331572 0.333005

有趣: 0.331424 0.333739 0.334837

--- 每个 query 词对应的输出向量范数 ---

机器学习: L2=0.143859

很: L2=0.143677

有趣: L2=0.143590

--- 按权重从高到低的前若干词对（query 行对 key 列） ---

1. query=很 对 key=机器学习 权重=0.335423 (i=1, j=0)
2. query=有趣 对 key=有趣 权重=0.334837 (i=2, j=2)
3. query=机器学习 对 key=机器学习 权重=0.333991 (i=0, j=0)
4. query=有趣 对 key=很 权重=0.333739 (i=2, j=1)
5. query=机器学习 对 key=很 权重=0.333475 (i=0, j=1)
6. query=很 对 key=有趣 权重=0.333005 (i=1, j=2)
7. query=机器学习 对 key=有趣 权重=0.332534 (i=0, j=2)
8. query=很 对 key=很 权重=0.331572 (i=1, j=1)

图表结果为：

无图表结果

5. 结果分析

1. 注意力权重矩阵表示什么含义？

矩阵第 i 行、第 j 列的数值，表示在第 i 个词作为 query 时，对第 j 个位置词向量经 K 映射后得到的 key 的归一化匹配强度。对每一行做 softmax 后，该行所有列之和为 1，可理解为当前词在构造新表示时，从全句各个位置上抽取信息的分配比例。

2. 哪些词之间的注意力权重较高？

对句子我爱自然语言处理，程序给出的前若干词对为，爱对我约 0.334920，自然语言处理对自然语言处理约 0.334523，我对自然语言处理约 0.334016，自然语言处理对爱约 0.333425，我对爱约 0.333283。

对句子机器学习很有趣，前若干词对为，很对机器学习约 0.335423，有趣对有趣约 0.334837，机器学习对机器学习约 0.333991，有趣对很约 0.333739，机器学习对很约 0.333475。

3. 自注意力如何体现“上下文依赖”？

每个位置的输出，都是对所有位置 V 的加权和，权重由当前位置与全句各位置的 Q 、 K 相似度决定。因此，同一个词在不同句子或不同邻接词环境下，会得到不同的加权和，其向量表示随上下文改变，这就是用分布式方式编码上下文依赖。

4. 为什么 Transformer 能处理长距离依赖？

与逐步压缩状态的 RNN 不同，自注意力在一步内建立任意两位置之间的直接打分路径，距离不会随间隔线性增加门控次数。只要层数与表示能力足够，远隔词仍可在注意力矩阵中获得非零权重，从而参与当前位置的表示更新。

5. 如果去掉 attention，会发生什么变化？

若完全去掉加权的上下文汇聚，仅保留位置自身的线性变换或前馈子层，则每个位置的表示与其他位置解耦，无法动态选择从句内何处读取信息，长距离依赖与指代、省略等现象难以建模。若形式上保留 V 但令注意力为均匀矩阵，则输出接近各位置 V 的平均，仍缺乏可学习的选择性。

6. 实验总结

本次实验从数据文件读入两条已空格分词的短句,为各词元构造可复现的词向量并经随机初始化的 Q、K、V 线性层,按公式计算缩放后的 QK 转置、行方向 softmax 得到注意力权重,再与 V 相乘得到各位置输出,打印每句的注意力矩阵、按权重排序的词对以及输出向量范数,程序正常跑通,便于对照任务书理解自注意力计算链。